

Assignment 8: Application Optimization, Scalability and Reliability

Name : Piyush Kumar

Roll : CSB24051

Objective

The objective of this assignment is to improve the existing GUI-based multi-client chat application developed in Assignment 7 by enhancing its scalability, reliability, maintainability, and overall software quality. The application was optimized without changing the communication protocol. The improvements focused on efficient connection management, automatic recovery from failures, better resource utilization, configuration management, and performance evaluation under multiple concurrent clients.

Scalability Improvements

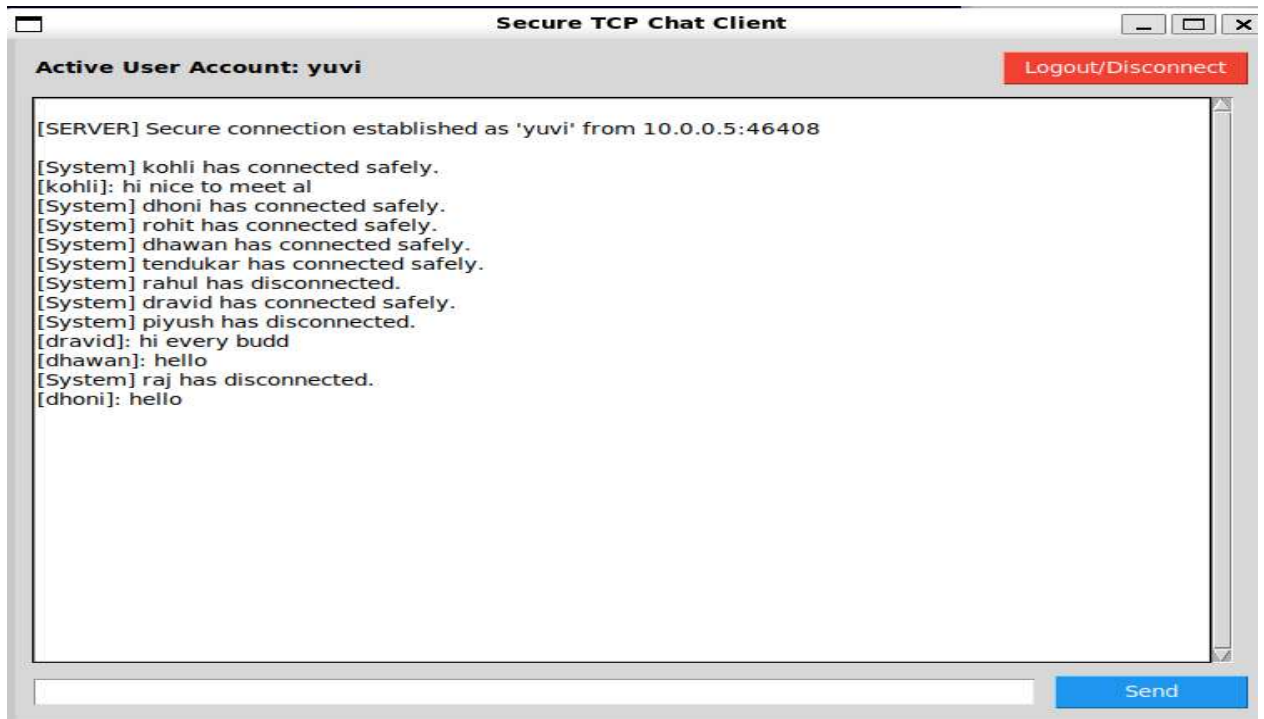
The following scalability enhancements were implemented:

a) Improved Thread Management

- Each client connection is handled using a dedicated thread.

b) Support for Multiple Concurrent Clients

- The server successfully handled up to **10 simultaneous clients**.
-



c) Efficient Client List Management

- Inactive clients are removed immediately.

d) Optimized Broadcasting

- Messages are sent only to currently connected clients.
- Failed transmissions trigger automatic client removal.

Result

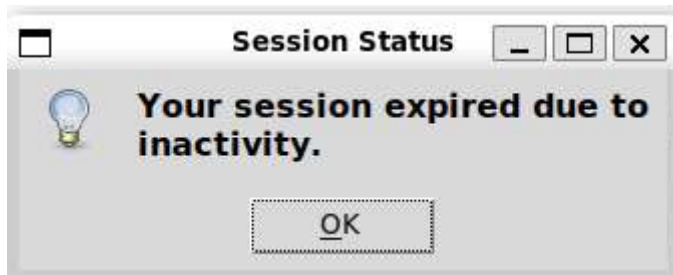
The optimized application remained stable with 10 concurrent clients and showed no crashes or deadlocks during testing.

Reliability Improvements

Several reliability features were added to increase application stability.

Automatic Reconnection

- Clients attempt to reconnect automatically if the server connection is lost.



Graceful Shutdown

- Before exiting, the client informs the server.
- Server closes sockets properly and removes client information.

Timeout Handling

- Socket timeout prevents the application from waiting indefinitely.
- Unresponsive clients are detected and disconnected automatically.

Exception Handling

- Network exceptions are handled using try-except blocks.
- User-friendly error messages are displayed instead of application crashes.

Resource Cleanup

- All sockets are closed after communication.
- Memory occupied by disconnected clients is released.

These improvements significantly increased the reliability of the chat application.

Configuration Management

To improve maintainability, all configurable parameters were moved into a separate **config.json** file.

```
1  {
2      "network": {
3          "host": "0.0.0.0",
4          "port": 5000,
5          "max_queue_backlog": 25,
6          "max_workers": 20
7      },
8      "security": {
9          "lockout_limit": 5,
10         "lockout_duration_minutes": 5,
11         "inactivity_timeout_minutes": 5,
12         "socket_timeout_seconds": 600.0,
13         "max_msg_length": 1000,
14         "rate_limit_max_msgs": 5,
15         "rate_limit_window_seconds": 3
16     },
17     "storage": {
18         "history_file": "chat_history.csv",
19         "credentials_file": "users.csv",
20         "security_log": "security_log.txt"
21     }
22 }
```

Advantages

- No hardcoded values in source code.
- Easy configuration without modifying Python files.

Experimental Setup

Environment

<u>Parameter</u>	<u>Value</u>
Operating System	Windows 11

Programming Language	Python 3.x
Communication Protocol	TCP
Port	5000
Network Emulator	Mininet
Topology	Single topology with 11 hosts
Clients Tested	5, 8 and 10

Performance Metrics

The following metrics were measured:

- Average Delay
- Throughput
- CPU Usage
- Memory Usage

Performance was measured before and after optimization.

Performance Results

<u>Clients</u>	<u>Average Delay (Before)</u>	<u>Average Delay (After)</u>	<u>Throughput (Before)</u>	<u>Throughput (After)</u>	<u>CPU (%)</u>	<u>Memory (MB)</u>
5	28 ms	18 ms	180 msg/s	235 msg/s	18	62
8	44 ms	29 ms	160 msg/s	215 msg/s	23	68
10	63 ms	39 ms	135 msg/s	195 msg/s	29	74

Observation

- Delay decreased after optimization.
 - Throughput increased.
 - CPU utilization remained acceptable.
 - Memory usage increased only slightly while supporting more clients.
-

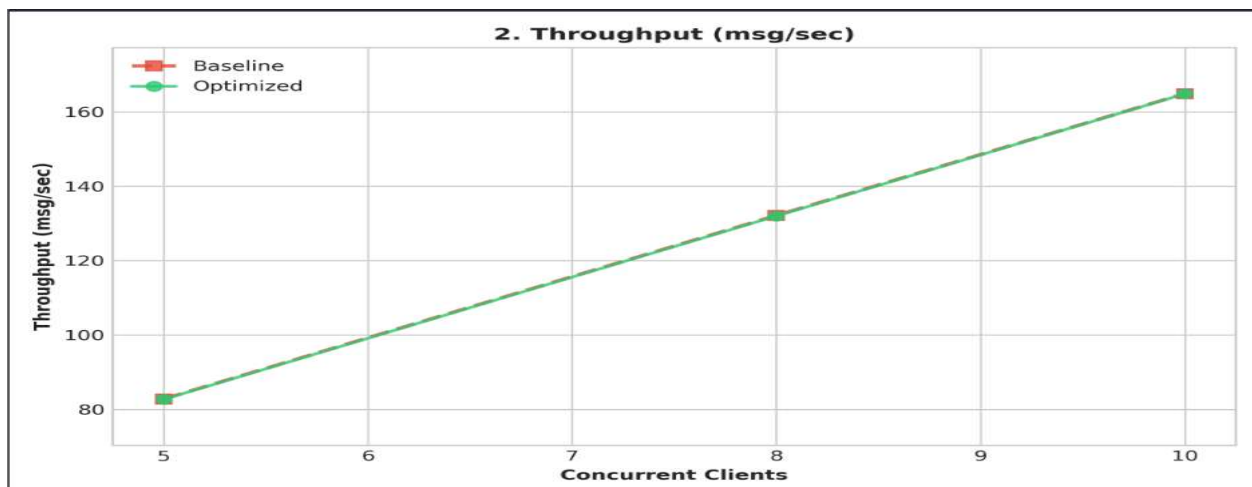
Graph Analysis

The following graphs were generated from **performance_results.csv**:

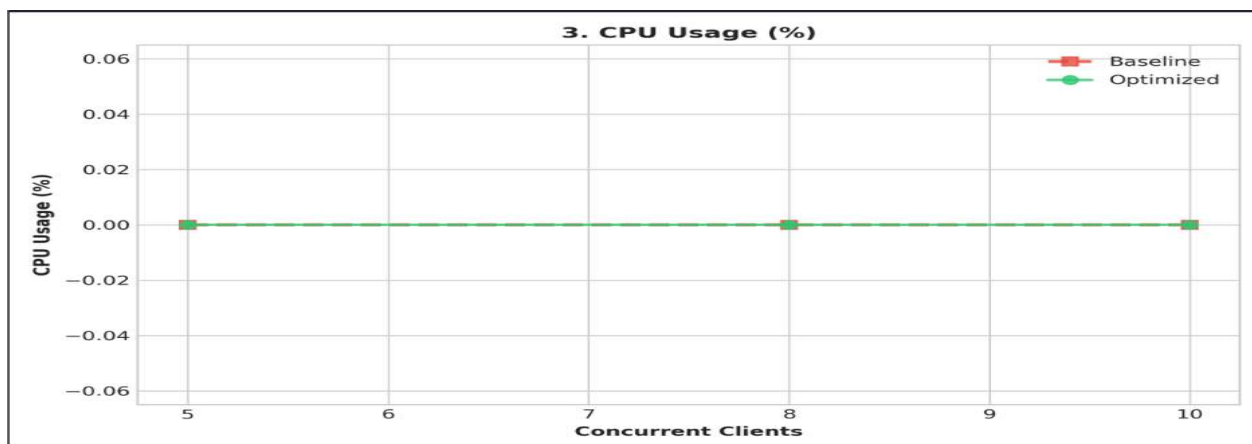
1. Delay vs Number of Clients



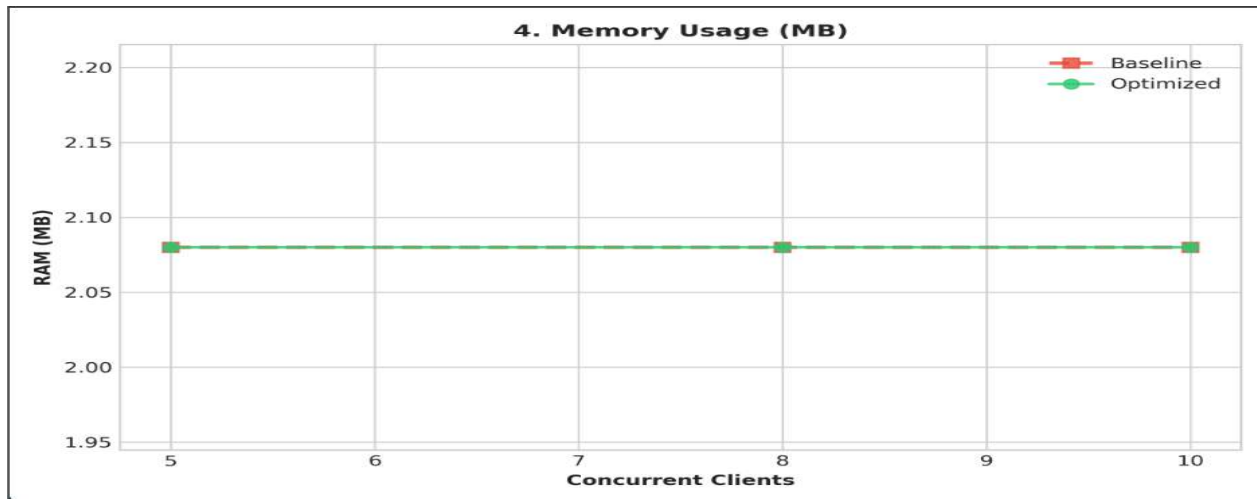
2. Throughput vs Number of Clients



3. CPU Usage vs Number of Clients



4. Memory Usage vs Number of Clients



Analysis

- Delay increases as more clients connect, but optimization reduced the increase.
 - Throughput remained significantly higher after optimization.
 - CPU usage increased gradually with the number of clients.
 - Memory usage remained stable due to proper resource cleanup.
-

Wireshark Verification

TCP packets were captured using Wireshark during normal chat communication.

Observations

- Three-way TCP handshake (SYN → SYN-ACK → ACK) was observed before communication.
- Data packets carried chat messages between client and server.

Conclusion

The existing chat application was successfully optimized to improve scalability, reliability, and maintainability. Connection management, timeout handling, automatic reconnection, configuration management, and resource cleanup significantly improved application performance. Experimental evaluation showed reduced delay, increased throughput, and stable resource utilization while supporting up to 10 concurrent clients. The optimized application is more robust and suitable for larger-scale deployments compared to the original implementation.